

Lecture Review

Symbol Table

API. A *symbol table* (or *map*) associates values with keys:

<code>put(key, val)</code>	insert (or update) the value associated with <code>key</code>
<code>get(key)</code>	return the value associated with <code>key</code>
<code>contains(key)</code>	is <code>key</code> in the table?
<code>remove(key)</code>	remove <code>key</code> and its value
<code>size()</code>	number of key–value pairs

Ordered symbol table. When keys come from an ordered universe (e.g., `int`, `std::string`), we can support richer queries:

<code>min()</code> / <code>max()</code>	smallest / largest key in the table
<code>floor(key)</code> / <code>ceiling(key)</code>	largest key $\leq$ / smallest key $\geq$ the given key
<code>rank(key)</code>	number of keys strictly smaller than the given key
<code>select(i)</code>	the key of rank <code>i</code>

Sets. A *set* is a symbol table with no associated values — only membership matters (`add(key)`, `contains(key)`, `remove(key)`). Every ordered-symbol-table operation above (`min`, `ceiling`, `rank`, `select`, ...) carries over unchanged, just without a value to return.

Elementary Implementations

- *Sequential search (unordered linked list).* Keep key–value pairs in a linked list, in no particular order. `get` and `put` both scan the list until the key is found or the end is reached: $\Theta(n)$ in the worst case for either operation.
- *Binary search (ordered array).* Keep keys sorted in an array. `contains` (and `ceiling`, `rank`) use *binary search*: maintain a search interval `[lo, hi]`, compare the key against the entry at the midpoint, and discard half the interval each step — $\Theta(\log n)$. `select(i)` is a single array access: $\Theta(1)$. But `add/remove` must shift every entry above the insertion/deletion point to keep the array sorted: $\Theta(n)$.

a[]	A	C	D	E	H	M	P	S
	0	1	2	3	4	5	6	7
	lo			mid				hi

Binary search for a key greater than E:

since `a[mid] = E` is too small, the interval narrows to `[mid + 1, hi]`.

Implementation	search	insert / delete
Sequential search (unordered linked list)	$\Theta(n)$	$\Theta(n)$
Binary search (ordered array)	$\Theta(\log n)$	$\Theta(n)$

Binary search wins on every query operation, which is exactly why Part A below asks you to build a set on top of a sorted array rather than a linked list — the $\Theta(n)$ cost of **add/remove** is unavoidable either way, but only the array gives you fast **contains**, **ceiling**, **rank**, and **select** for free.

Lab Exercises

Part A: Set implementation using ordered-array

Implement the set data type using an ordered array. That is, you maintain the n keys in the set in ascending order in an array. Implement the following operations with the specified worst-case time complexity:

Use the code in `set.hpp` as a starting point. It declares the `Set<Key>` class with `keys_` already stored as a sorted `std::vector<Key>`; complete the private `binarySearch` helper first (it locates the position of a key, or where it belongs), then use it to implement **contains**, **ceiling**, **rank**, **select**, **min**, **add**, and **remove**.

<code>add(key)</code>	<i>add the key to the set</i>	$\Theta(n)$
<code>contains(key)</code>	<i>is the key in the set?</i>	$\Theta(\log n)$
<code>ceiling(key)</code>	<i>smallest key in set $\geq$ given key</i>	$\Theta(\log n)$
<code>rank(key)</code>	<i>number of keys in set $<$ given key</i>	$\Theta(\log n)$
<code>select(i)</code>	<i>i-th largest key in the set</i>	$\Theta(1)$
<code>min()</code>	<i>minimum key in the set</i>	$\Theta(1)$
<code>remove(key)</code>	<i>delete the given key from the set</i>	$\Theta(n)$

Note:

- You may use `std::vector` for resizing support.
- Do not use binary search from standard library. Implement your own binary search for **contains**, **ceiling**, and **rank** operations.
- For **add** and **remove**, you may need to shift the elements in the array. Do not use vector's **insert** and **erase** functions.
- Make sure that set contains unique keys.

Use `test-set.cpp` to check your implementation: it builds a `Set<char>`, exercises every operation above, and prints each result next to the expected value as a comment.

Part B: Exam room (Leet code 855)

There is an exam room with n seats in a single row labeled from 0 to $n - 1$. When a student enters the room, they must sit in the seat that maximizes the distance to the closest person. If there are multiple such seats, they sit in the seat with the lowest number. If no one is in the room, then the student sits at seat number 0.

Implement the `ExamRoom` class that simulates the mentioned exam room.

Use the code in `examroom.cpp` as a starting point. It already declares `occupied_` as a `std::set<int>` and includes a `main()` that reproduces the example below for testing; complete `seat()` and `leave(p)`.

- `ExamRoom(int n)`
Initializes the object of the exam room with the number of the seats n .
- `int seat()`
Returns the label of the seat at which the next student will set.
- `void leave(int p)`
Indicates that the student sitting at seat p will leave the room. It is guaranteed that there will be a student sitting at seat p .

Example:

```
ExamRoom examRoom(10);
examRoom.seat(); // return 0, no one is in the room, then the student sits at seat number 0.
examRoom.seat(); // return 9, the student sits at the last seat number 9.
examRoom.seat(); // return 4, the student sits at the last seat number 4.
examRoom.seat(); // return 2, the student sits at the last seat number 2.
examRoom.leave(4);
examRoom.seat(); // return 5, the student sits at the last seat number 5.
```

Hint: The problem requires us to seat students in such a way that they are seated as far as possible from the already seated students. The challenge lies in efficiently finding the optimal seat given potentially many occupied seats while ensuring that we can update our data structure when a student leaves. The key observation is that we can treat the empty seats as segments/gaps and determine the best seat to occupy in those segments based on the distance from the occupied seats.

We use a set from previous exercise or `std::set` from the standard library to maintain a list of occupied seats. This allows us to quickly check which seats are taken and to remove seats efficiently when students leave.

Seating Logic:

- If the room is empty, the first student sits at seat 0.
- If there are already occupied seats, we:
 - Calculate the distance from the start of the row (seat 0) to the first occupied seat.
 - Check the gaps between each pair of occupied seats to find the maximum distance.
 - Finally, calculate the distance from the last occupied seat to the end of the row (seat $n - 1$).
- The seat that provides the maximum distance from the occupied seats among all empty segments is chosen .

Leaving Logic: When a student leaves, we simply remove their seat from the set of occupied seats.